

实验 7：多周期 CPU 设计与测试

一、实验目的

1. 掌握 RV32I 多周期 CPU 数据通路设计方法。
2. 掌握多周期 CPU 中控制器状态机设计方法。
3. 掌握多周期 CPU 测试程序的设计方法。

二、实验环境

Logisim: <https://github.com/Logisim-Ita/Logisim>

RISC-V 模拟器工具 RARS: <https://github.com/thethirdone/rars>

三、实验内容

单周期处理器时钟周期取最复杂指令所用指令周期，因而时钟周期远远大于许多指令实际所需执行时间。由于受时钟周期宽度的影响，程序执行时间长，处理器的性能低，在实际应用很少采用单周期方式的设计 CPU，而采用多周期或流水线的方式设计 CPU。

多周期 CPU 的设计方式是把指令的执行分成多个阶段，每个阶段在一个或多个时钟周期内完成；每个时钟周期称为一个状态，期间最多完成一次访存或一次寄存器读写或一次 ALU 操作；每个时钟内的执行结果在下个时钟信号到来前，都保存到相应存储元件或稳定地保持在组合电路中；时钟周期的宽度以最复杂阶段所用的时间为准。和单周期处理器最主要的差异在于，每条指令的执行都是分步完成的。每一步得到的结果都存储在寄存器中。通过这种方法，一条指令可以在不同时钟周期中多次使用同一个功能部件，例如 ALU 可以在不同的周期中重复使用；同样，指令和数据也可以存储在同一个存储器中，这种共享可以减少所需硬件的数量。

在多周期 CPU 中需要考虑总线、指令/数据存储器、三态门等部件在数据通路的实现以及根据状态机生成指令的控制信号，并能考虑中断和异常状态的实现。

1、多周期 CPU 控制器设计

分析 RV32I 指令的执行过程，一般可以分成取指令 IF、取操作数(和指令译码)ID、指令执行 Exec 和完成 Finish 等阶段。每个时钟周期执行一个阶段，执行一条指令需要多个阶段，也就需要多个时钟周期。不同指令的执行阶段数不同，需要的时钟周期数也不同，复杂指令需要的时钟周期多，简单指令需要时钟周期少。不同执行阶段之间通过寄存器来传递数据，通过三态门来选择地址总线上传输的是指令地址还是数据地址。增加一个状态寄存器来存储 ALU 执行后的状态标志位。

多周期 CPU 设计需要增加五个寄存器：指令寄存器 IR、存储器地址寄存器 MDR、ALU 结果寄存器 ALUOut、存储器数据寄存器 MDR、状态寄存器 CC；增加两个三态门控制信号，互斥传送寄存器 PC 和存储器地址寄存器 MAR 中的数据到地址总线上；所有寄存器和存储器都需要写使能控制信号 W_r ，存储器使用片选控制信号 En 。多周期 CPU 的设计原理图如图 7-1 所示。

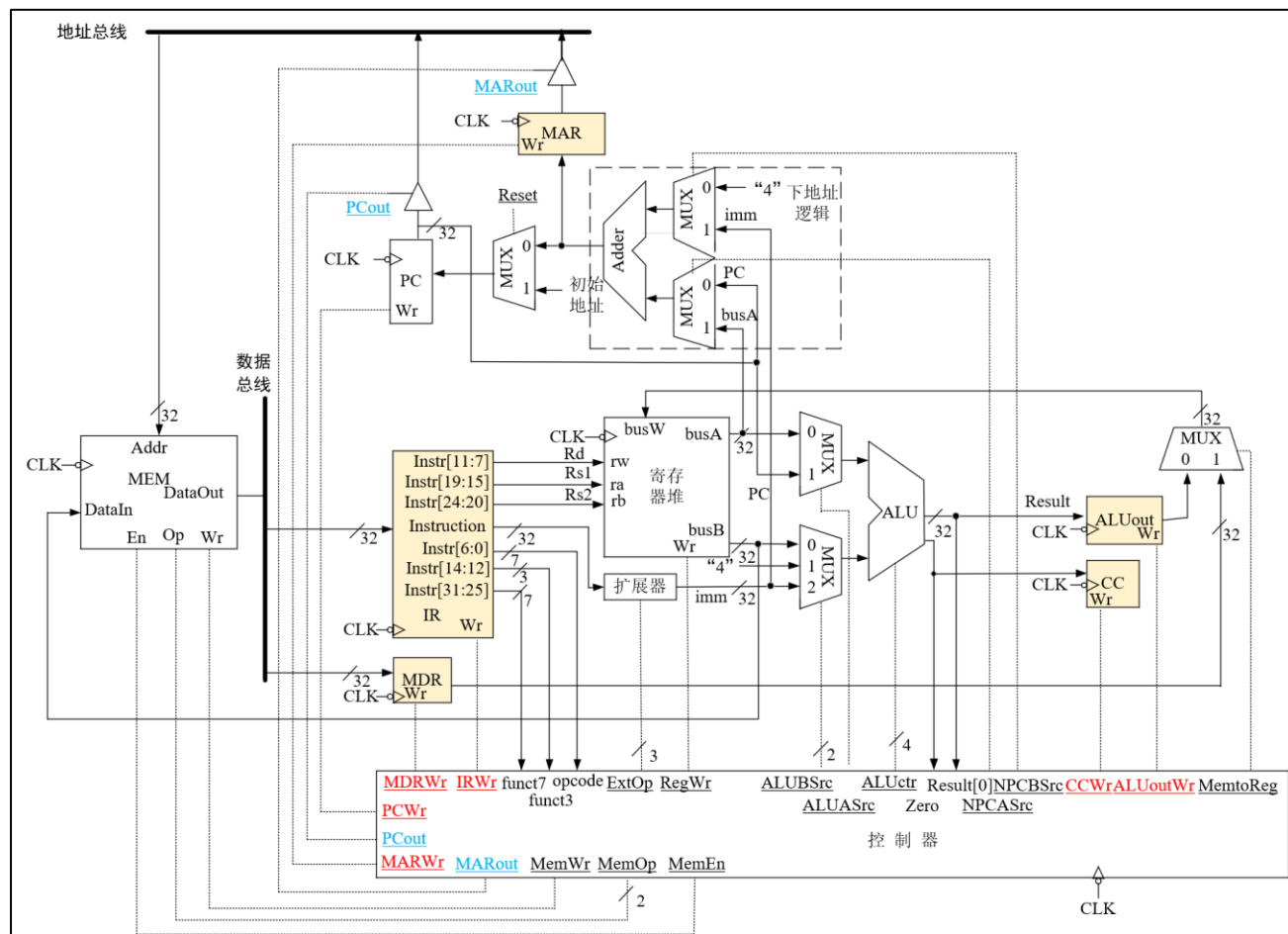


图 7-1 多周期 CPU 设计原理图

与单周期 CPU 相比，多周期 CPU 在取指令阶段将计算下条指令地址： $PC+4$ ，把结果写入到程序计数器 PC 中；在指令译码阶段将进行存储器地址的“投机计算”，利用加法器计算 $R[Rs1]+SEXT[imm]$ ，然后把结果写入到存储器地址寄存器 MAR 中，因此在这两个阶段需要直接给控制信号 NPCASrc 和 NPCBSrc 赋值，在指令执行阶段，NPCASrc 和 NPCBSrc 根据 B 型指令及指令执行结果进行赋值。为了能在取指和译码取数阶段直接得到 NPCASrc 和 NPCBSrc 的数值，在指令控制器中取消了 BandJ 控制信号。

由于使用同一个存储器来存储程序和数据，为了区分存储器输入的指令地址还是数据地址，三态门控制信号 PCout 和 MARout 需要在不同的指令执行阶段分别有效。同样，增加的不同功能的寄存器也只能在不同指令执行阶段，分别设置对应的写使能信号有效。写使能信号都在时钟信号下降沿有效。

下面介绍不同类型指令功能以及在指令执行阶段过程中的控制信号取值。

（1）取指令并计算下条指令地址 IF

在取指令阶段 IF 功能是：将 PC 寄存器中的数据作为地址访问主存以取出指令，将指令写入指令寄存器

IR中，并将PC+4作为下条指令地址送PC。因为采用定长指令字，故PC增量操作可在取指令阶段完成。由于通过PC寄存器传输指令地址到地址总线，因此PC寄存器输出端的三态门控制信号PCOut赋值为高电平有效，读取存储器中的指令，把数据写入到指令寄存器中IR中，IRWr需有效；同时存储器地址寄存器输出端的三态门控制信号MARout赋值为低电平无效。为了利用下地址逻辑计算下条指令地址，需赋值NPCASrc=0和NPCBSrc=0，同时把结果写入寄存器PC中，PCWr需有效。

需要注意的是，如果在取指令阶段就把PC+4写入PC寄存器中，在B型和J型指令的后续阶段使用到PC值时，实际上是PC+4，因此需要先执行 $PC \leftarrow PC-4$ ，再使用PC值，导致产生额外的硬件支撑和时间开销。为简化设计，在实验中计算下条指令更新PC值的阶段，统一到指令执行阶段，而在取指令阶段不写入PC寄存器，PCWr=0。

实现上述功能的有效控制信号及其取值如下。

① $R[IR] \leftarrow M[PC]$: PCOut=1、IRWr=1、MARout=0、MemOp=000、MemEn=1。

② 其他寄存器写使能信号（如MARWr、MemWr、MDRWr、ALUoutWr、RegWr、CCWr）全部为0。

（2）译码并取数ID

在指令译码和取操作数阶段的功能是，根据指令寄存器 IR 中的输出内容，分解指令字段，根据指令操作码、功能码字段被送到控制部件中的指令译码器进行译码；rs1 和 rs2 字段分别送到寄存器堆的 Ra 和 Rb 输入端，经过一个“取数时间”，在 BusA 和 BusB 端分别输出 Rs1、Rs2 寄存器的内容。

指令寄存器 IR 中的指令被分解操作码和功能码送到控制器生成 ExtOp、ALUASrc、ALUBSrc、ALUctr、MemtoReg 和 MemOp 等控制信号的数值，立即数控制扩展器根据控制信号 ExtOp 的数值该指令的输出 32 位立即数。

该阶段下地址逻辑的加法器是空闲的，可以利用加法器进行存储器地址“投机”计算，计算存储器地址 $R[rs1]+imm$ ，并将结果暂存在 MAR 中。

实现上述功能的有效控制信号及其取值如下。

① $ExtOp=xxx$ ，ALUASrc=x、ALUBSrc=xx、ALUctr=xxxx。

② $R[MAR] \leftarrow R[Rs1]+imm$: NPCASrc=1、NPCBSrc=1、MARWr=1、MARout=0、PCOut=0。其他寄存器写使能信号（如PCWr、MemWr、MDRWr、ALUoutWr、RegWr、CCWr）全部为0。

（3）执行指令EXE

不同类型指令的功能不同，执行阶段需要的时钟周期数不同，涉及的控制信号也不同，具体情况如下。

1) R-型指令

R-型指令包括R-型运算类指令和Jalr指令。

R-型运算类指令的功能是： $R[Rd] \leftarrow R[Rs1] \text{ op } R[Rs2]$ 。

这类R型指令的执行需要两个时钟周期，对应状态分别记为RExec和RFinish。

RExec状态的功能为：根据ALUASrc和ALUBSrc选择操作数的来源，在ALUctr信号控制下进行ALU运算并将结果存入寄存器ALUout，标志位Zero写入状态寄存器CC。其有效控制信号及其取值为：ALUoutWr=1、CCWr=1，其他寄存器写使能信号全部为0。此外，在该阶段计算下条指令地址 $PC \leftarrow PC+4$ ，控制信号赋值

如下：NPCASrc=0、NPCBSrc=0、PCWr=1。

RFinish状态的功能为：将ALUout的内容写入寄存器堆中寄存器Rd。其有效控制信号及其取值：MemtoReg=0，RegWr=1，其他寄存器写使能信号全部为0。

Jalr指令功能是： $R[Rd] \leftarrow PC + 4$ ； $R[PC] \leftarrow R[Rs1] + imm$ 。指令执行需要两个加法运算，分别对应下地址逻辑中的加法和ALU部件，执行需要两个时钟周期，对应状态分别记为JalrExec和JalrFinish。

JalrExec状态的功能为：通过ALU计算PC+4，将结果写入寄存器ALUout和状态寄存器CC。在译码阶段已经投机计算R[Rs1]+imm，将结果写入PC寄存器。其有效控制信号及其取值为：ALUoutWr=1、CCWr=1、NPCASrc=1、NPCBSrc=1、PCWr=1，其他寄存器写使能信号全部为0。

JalrFinish状态的功能为：将ALUout的内容写入寄存器堆中寄存器Rd。其有效控制信号及其取值：MemtoReg=0，RegWr=1，其他寄存器写使能信号全部为0。

2) I-型运算指令

I-型指令包括I-型运算类指令和lode类型指令。

I-型运算类指令的功能是： $R[Rd] \leftarrow R[Rs1] \text{ op EXT}[imm]$

这些I型运算指令的执行需要两个时钟周期，对应状态分别记为IExec和IFinish。

IExec状态的功能为：根据ALUASrc和ALUBSrc选择操作数的来源，在ALUctr信号控制下进行ALU运算并将结果存入寄存器ALUout，标志位Zero写入状态寄存器CC。其有效控制信号及其取值为：ALUoutWr=1、CCWr=1，其他寄存器写使能信号全部为0。此外，在该阶段计算下条指令地址 $PC \leftarrow PC + 4$ ，控制信号赋值如下：NPCASrc=0、NPCBSrc=0、PCWr=1。

IFinish状态的功能为：将ALUout的内容写入寄存器堆中寄存器Rd。其有效控制信号及其取值：MemtoReg=0，RegWr=1，其他寄存器写使能信号全部为0。

Load类型指令是一种特殊的I型指令，其功能是： $R[Rd] \leftarrow M[R[Rs1] + \text{SEXT}[imm]]$ ，指令执行过程包含：计算数据在存储器中地址R[Rs1] + SEXT[imm]、读取该存储器地址中的数据、写入到指定的寄存器Rd中三个阶段。由于在“译码并取数”阶段已经计算出地址并存入MAR中，因而还需要两个时钟周期，对应状态分别记为LExec和LFinish。

LExec状态的功能为：根据MemOp信号读存储器内容并保存到MDR。其有效控制信号及其取值为：MARout=1、PCout=0、MemWr=0、MDRWr=1、MemEn=1，其他寄存器写使能信号全部为0。此外，在该阶段计算下条指令地址 $PC \leftarrow PC + 4$ ，控制信号赋值如下：NPCASrc=0、NPCBSrc=0、PCWr=1。

LFinish状态的功能为：将MDR内容存入Rd。其有效控制信号及其取值为：MemtoReg=1、RegWr=1，其他寄存器写使能信号全部为0。

3) S-型指令

S型指令的功能是： $M[R[Rs1] + \text{SEXT}[imm]] \leftarrow R[Rs2]$ 。指令执行过程包含计算存储器地址R[Rs1] + SEXT[imm]和读取Rs2寄存器中的数据、写入到存储器指定地址中二个阶段，需要二个时钟周期。因为在“译码并取数”阶段已经计算出地址并存入MAR中，同时已经取出Rs2寄存器中数据在BusB总线上，因而只需要1个时钟周期，对应状态记为SFinish。

SFinish状态的功能为：根据MemOp信号将BusB总线的数据写入主存指定地址中。其有效控制信号及其取值为：MARout=1、PCout=0、MemWr=1、MemEn=1，其他寄存器写使能信号全部为0。

此外，在该阶段计算下条指令地址 $PC \leftarrow PC+4$ ，控制信号赋值如下：NPCASrc=0、NPCBSrc=0、PCWr=1。

4) B-型指令

B型指令的功能是比较R[Rs1]和R[Rs2]的大小关系，如果关系成立，则 $PC \leftarrow PC + \text{SEXT}[imm]$ ，否则 $PC \leftarrow PC + 4$ 。指令的执行需要一个时钟周期，对应状态记为BFinish。

BFinish状态的功能为：通过减法比较R[Rs1]和R[Rs2]的大小，生成等于状态位Zero和小于状态位Result[0]，并把状态位写入到状态寄存器CC中；根据指令操作码、功能码、Zero标志位以及ALU计算结果的最低位Result[0]，生成NPCASrc和NPCBSrc值，通过下地址逻辑计算下条指令的地址，然后写入PC寄存器中。其有效控制信号及其取值为：PCWr=1、PCout=0、CCWr=1，其他寄存器写使能信号全部为0。

5) J-型指令

Jump指令的功能为： $PC \leftarrow PC + \text{SEXT}[imm]$ ， $R[Rd] \leftarrow PC + 4$ 。指令执行需要两个加法运算，分别对应下地址逻辑中的加法器和ALU部件，执行需要两个时钟周期，对应状态分别记为JExec和JFinish。

JExec状态的功能为：通过ALU计算 $PC+4$ ，将结果写入寄存器ALUout和状态寄存器CC。在译码阶段已经投机计算 $PC+imm$ ，将结果写入PC寄存器。其有效控制信号及其取值为：ALUoutWr=1、CCWr=1、NPCASrc=0、NPCBSrc=1、PCWr=1，其他寄存器写使能信号全部为0。

JFinish状态的功能为：将ALUout的内容写入寄存器堆中寄存器Rd。其有效控制信号及其取值：MemtoReg=0，RegWr=1，其他寄存器写使能信号全部为0。

6) U-型指令

lui指令的功能是 $R[Rd] \leftarrow imm \ll 12$ ；auipc指令的功能是 $R[Rd] \leftarrow PC + imm \ll 12$ 。

U型指令的执行需要两个时钟周期，对应状态分别记为UExec和UFinish。

UExec状态的功能为：根据ALUASrc和ALUBSrc选择操作数的来源，在ALUctr信号控制下进行ALU运算并将结果存入寄存器ALUout，标志位Zero写入状态寄存器CC。其有效控制信号及其取值为：ALUoutWr=1、CCWr=1，其他寄存器写使能信号全部为0。此外，在该阶段计算下条指令地址 $PC \leftarrow PC+4$ ，控制信号赋值如下：NPCASrc=0、NPCBSrc=0、PCWr=1。

UFinish状态的功能为：将ALUout的内容写入寄存器堆中寄存器Rd。其有效控制信号及其取值：MemtoReg=0，RegWr=1，其他寄存器写使能信号全部为0。

通过分析，可以看出RExec、IExec、UExec三个状态的功能及控制信号完成相同，可以合并成一个状态RIUExec。RFinish、IFinish、JalrFinish、UFinish、JFinish三个状态的功能及控制信号完成相同，可以合并成一个状态RIUJFinish。

根据上述对每条指令执行过程的分析，得到一个状态转换图。图 7-2 是一个支持 RV32I 指令执行的状态转换示意图。图中每个状态用一个状态编号和状态名标识，例如，S0:IFetch 表示第 0 状态，执行取指令（IFetch）操作，圆圈中示意性地给出了该状态下部分控制信号的取值，其中，有取值为 0 和取值为 1 的两

种有效控制信号，以及多值有效控制信号 $ExtOp=xxx$ 表示根据指令操作码译码得到的一个立即数扩展器控制信号取值为 xxx 。

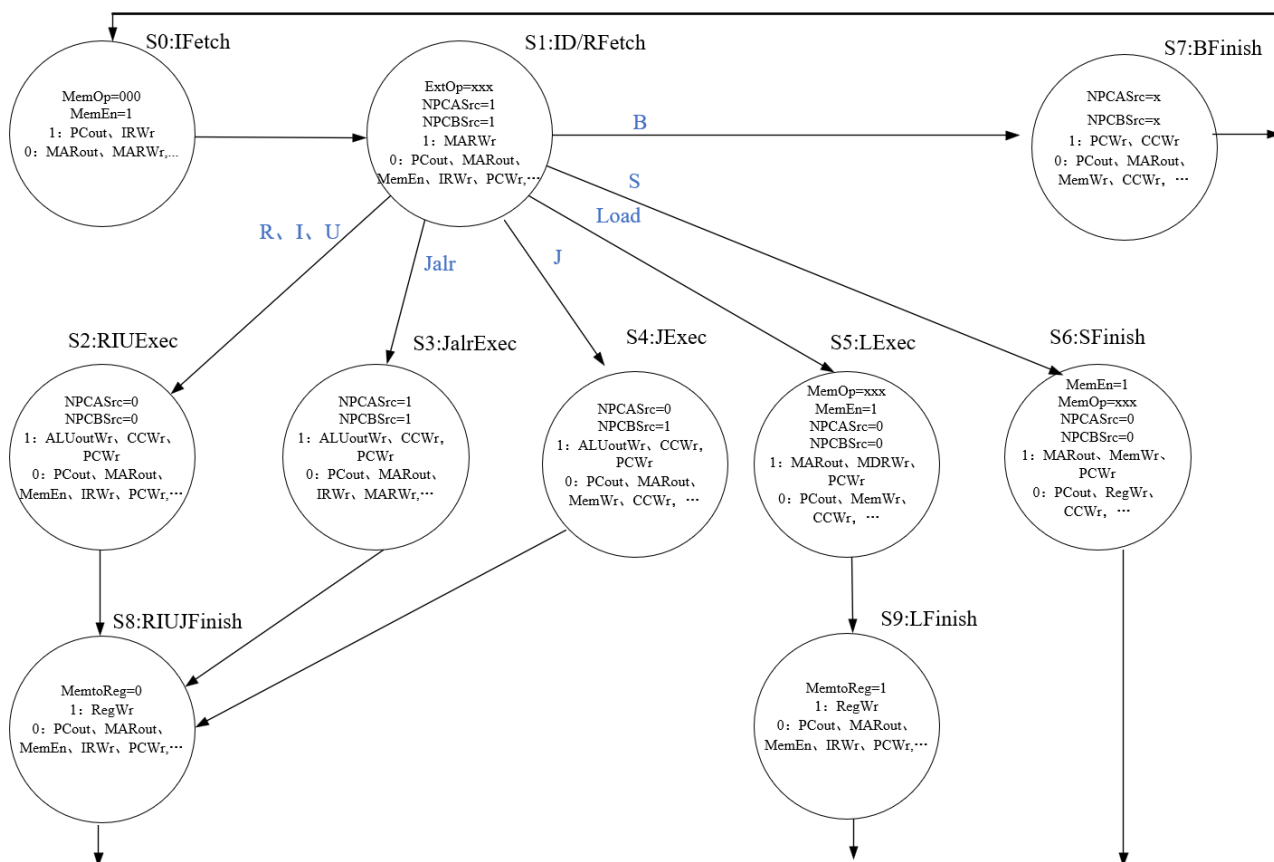


图 7-2 指令执行状态转换图

在图 7-1 所示的多周期 CPU 设计原理图中，每条指令的执行过程就是图 7-2 所示的状态转换过程。每条指令经历一个时钟周期后，进入下一个状态。从图中可看出，R、I、U、J 型指令的执行周期都是 4 个时钟周期，而 S 和 B 型指令的执行周期是 3 个时钟周期。如果不在译码/取数阶段“投机”计算访存地址，则 Load 指令的 CPI 为 5，S 型指令的 CPI 为 4。

与单周期 CPU 相比，多周期 CPU 中增加了 5 个寄存器以及 2 个三态门等器件。此外，PC 寄存器不是在每个时钟周期都需要修改，因此需要增加写使能控制 $PCWr$ 。多周期需要的控制信号如下所示，这些控制信号定义的具体含义是：

$PCWr$ ：PC 寄存器写使能控制信号，根据指令执行阶段来赋值，高电平有效。

$MARWr$ ：存储器地址寄存器 MAR 写使能控制信号，根据指令执行阶段来赋值，高电平有效。

$IRWr$ ：指令寄存器写使能控制信号，根据指令执行阶段来赋值，高电平有效。

$MDRWr$ ：存储器数据寄存器 MDR 写使能控制信号，根据指令执行阶段来赋值，高电平有效。

$ALUoutWr$ ：ALU 输出寄存器写使能控制信号，根据指令执行阶段来赋值，高电平有效。

$CCWr$ ：状态寄存器 CC 写使能控制信号，根据指令执行阶段来赋值，高电平有效。

$RegWr$ ：寄存器堆写使能控制信号，根据指令类型在相应的执行阶段来赋值，高电平有效。

$MemWr$ ：数据存储器写使能控制信号，高电平有效。

MemEn: 存储器片选控制信号, 根据指令执行阶段来赋值, 高电平有效。

PCout: PC 寄存器输出三态门控制信号, 根据指令执行阶段来赋值, 高电平有效。

MARout: 存储器地址寄存器 MAR 输出三态门控制信号, 根据指令执行阶段来赋值, 高电平有效。

MemtoReg: 寄存器堆写入数据来源控制信号, 0: ALU 运算结果, 1: 数据存储器读取输出。

ExtOp: 立即数产生器的控制信号, 根据指令类型赋值, 宽度为 3 位。

ALUASrc: ALU 输入端 A 的选择信号, 0: R[Rs1], 1: PC。

ALUBSrc: ALU 输入端 B 的选择信号, 宽度为 2 位, 00: R[Rs2], 01: 4, 10: imm。

ALUctr: ALU 控制信号, 根据指令操作码和功能码赋值, 宽度为 4 位。

NPCASrc: 下地址逻辑中加法器输入端 A 的选择信号, 0: PC, 1: R[Rs1]。

NPCBSrc: 下地址逻辑中加法器输入端 B 的选择信号, 0: 4, 1: imm。

MemOp: 数据存储器读写操作以及字节长度控制信号, 宽度为 3 位。

在这些多周期控制信号中, 只和指令执行阶段有关的控制信号包括写使能信号: PCWr、MARWr、IRWr、MDRWr、ALUoutWr、CCWr、RegWr、MemWr、MemEn、MemtoReg 以及三态门控制信号 PCout 和 MARout。这些控制信号和指令执行状态的定义如表 7-1 所示。

表 7-1 与指令执行阶段相关的控制信号定义

状态编号 控制信号	S0 IF	S1 ID/RF	S2 RIUExec	S3 JalrExec	S4 JExec	S5 LExec	S6 SFinish	S7 BFinish	S8 RIUJFinish	S9 LFinish
PCout	1	0	0	0	0	0	0	0	0	0
MARout	0	0	0	0	0	1	1	0	0	0
PCWr	0	0	1	1	1	1	1	1	0	0
MARWr	0	1	0	0	0	0	0	0	0	0
IRWr	1	0	0	0	0	0	0	0	0	0
MDRWr	0	0	0	0	0	1	0	0	0	0
ALUoutWr	0	0	1	1	1	0	0	0	0	0
CCWr	0	0	1	1	1	0	0	1	0	0
MemWr	0	0	0	0	0	0	1	0	0	0
RegWr	0	0	0	0	0	0	0	0	1	1
MemEn	1	0	0	0	0	1	1	0	0	0
MemtoReg	0	0	X	0	0	0	0	0	0	1

只与指令操作码功能码有关的控制信号包括: ExtOp、ALUASrc、ALUBSrc、ALUctr, 这些控制信号的赋值和单周期 CPU 控制器设计中相同, 具体定义如表 7-2 所示。

表 7-2 ExtOp、ALUASrc、ALUBSrc、ALUctr 控制信号定义表

指令	类型	op[6:0]	func3	func7[5]	ExtOp	ALUASrc	ALUBSrc	ALUctr
lui	U	0110111	×	×	001	×	10	1111
auipc	U	0010111	×	×	001	1	10	0000
addi	I	0010011	000	×	000	0	10	0000
slti	I	0010011	010	×	000	0	10	0010

sltiu	I	0010011	011	×	000	0	10	0011
xori	I	0010011	100	×	000	0	10	0100
ori	I	0010011	110	×	000	0	10	0110
andi	I	0010011	111	×	000	0	10	0111
slli	I	0010011	001	0	000	0	10	0001
srli	I	0010011	101	0	000	0	10	0101
srai	I	0010011	101	1	000	0	10	1101
add	R	0110011	000	0	×	0	00	0000
sub	R	0110011	000	1	×	0	00	1000
sll	R	0110011	001	0	×	0	00	0001
slt	R	0110011	010	0	×	0	00	0010
sltu	R	0110011	011	0	×	0	00	0011
xor	R	0110011	100	0	×	0	00	0100
srl	R	0110011	101	0	×	0	00	0101
sra	R	0110011	101	1	×	0	00	1101
or	R	0110011	110	0	×	0	00	0110
and	R	0110011	111	0	×	0	00	0111
jal	J	1101111	×	×	100	1	01	0000
jalr	I	1100111	000	×	000	1	01	0000
beq	B	1100011	000	×	011	0	00	0010
bne	B	1100011	001	×	011	0	00	0010
blt	B	1100011	100	×	011	0	00	0010
bge	B	1100011	101	×	011	0	00	0010
bltu	B	1100011	110	×	011	0	00	0011
bgeu	B	1100011	111	×	011	0	00	0011
lb	I	0000011	000	×	000	0	10	0000
lh	I	0000011	001	×	000	0	10	0000
lw	I	0000011	010	×	000	0	10	0000
lbu	I	0000011	100	×	000	0	10	0000
lhu	I	0000011	101	×	000	0	10	0000
sb	S	0100011	000	×	010	0	10	0000
sh	S	0100011	001	×	010	0	10	0000
sw	S	0100011	010	×	010	0	10	0000

而 MemOp、NPCASrc 和 NPCBSrc 这些信号不但与指令执行阶段有关也和指令操作码及执行结果有关。存储器读写操作以及字节长度控制信号 MemOp 的取值定义如表 7-3 所示。

表 7-3 MemOp 控制信号含义

指令执行阶段	MemOp	指令类型	含 义
取指令 IF	000		按字读取，4 字节
Load 和 Store 指令执行阶段	000	lw,sw	按字存取，4 字节
	001	lbu	按字节读取，1 个字节，0 扩展到 4 字节

	010	lhu	按半字读取, 2 字节, 0 扩展到 4 字节
	101	lb, sb	按字节存取, 1 个字节, 在读取时, 按符号位扩展到 4 字节
	110	lh, sh	按半字存取, 2 个字节, 在读取时, 按符号位扩展到 4 字节

在多周期 CPU 设计中, 取消了分支跳转控制部件, 过程控制信号 **BandJ**, 直接通过指令类型、执行阶段和 ALU 执行结果来生成下地址加法器输入端的选择信号 **NPCASrc** 和 **NPCBSrc**, 其赋值定义如表 7-4 所示。

表 7-4 下地址逻辑加法器输入端控制信号

指令执行阶段	指令类型	NPCASrc	NPCBSrc
取操作数指令译码		1	1
分支和跳转指令执行阶段	非跳转指令	0	0
	jal: 无条件跳转 PC 目标	0	1
	jalr: 无条件跳转寄存器目标	1	1
	beq: 条件分支, 等于	0	Zero
	bne: 条件分支, 不等于	0	! Zero
	blt, bltu: 条件分支, 小于	0	Result[0]
	bge, bgeu: 条件分支, 大于等于	0	Zero (! Result[0])
B 型和 S 型指令结束阶段		0	0

由于多周期数据通路中每条指令的执行有多个时钟周期, 每个时钟周期的控制信号取值不同, 所以, 不能像设计单周期控制器那样用简单组合电路来实现。多周期控制器通常采用基于有限状态机描述和微程序描述两种方式来实现。

有限状态机描述方式实现的控制器称为有限状态机控制器, 其基本思想为: 用一个有限状态机描述指令执行过程, 由当前状态和操作码确定下一状态, 每来一个时钟发生一次状态改变, 不同状态输出不同的控制信号值, 然后送到数据通路相应的控制点, 以控制数据通路中数据的运算、存储和传送。对于图 7-2 所示的状态转换图, 假定每个状态编号如图中所设, 分别为 0~9, 共 10 个状态, 因此, 状态变量至少需要 4 位。

图 7-3 描述了采用这种方式实现的控制器结构, 它由两部分组成: 一个组合逻辑控制单元和一个由状态寄存器构成确定性有限状态机 (Deterministic Finite Automaton, DFA) 单元, 可用 PLA 电路实现组合逻辑控制单元。

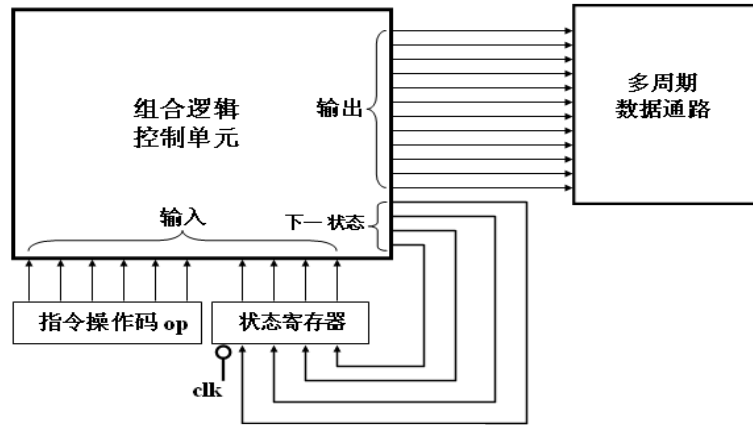


图 7-3 有限状态机控制器结构

为了便于检测程序执行的结果和异常指令，在控制器中额外增加了两个输出信号中断执行信号 `ecall` 和非法指令信号 `invalid`，当指令操作码 `opcode=0x73` 时，`ecall` 赋值为 1，PC 寄存器停止执行，PC 保持不变，中止程序的执行。当指令操作码 `opcode` 不属于 RV32I 中的有效指令时，`invalid` 赋值为 1，表示程序中包含非法指令，将中止 PC 寄存器的执行。在调试程序时，可以通过插入 `ecall` 指令或非法指令编码来中止程序执行，察看执行结果。

实验步骤如下：

1) 设计控制器电路。在 Logisim 中打开实验模版文件 `lab7.1.circ`，其引脚布局图如 7-4 所示，布局图中根据不同类型指令操作码通过比较器来判别指令类型，实验时也可以通过与门来实现。在设计控制器电路时，基于 PLA 设计架构，根据指令种类和状态编码，生成控制信号，可分成与指令类型相关、与指令状态相关、和指令类型及指令状态都相关、状态机等多个情形来设计。在主电路下，根据表 7-2 利用 PLA 电路生成 `ExtOp`、`ALUASrc`、`ALUBSrc`、`ALUctr`、`invalid` 等控制信号设计电路。根据图 7-2 实现确定性有限状态机 DFA，状态 `S0~S9` 分别赋值为 `0000~1001`，状态信号设置为 `Q3Q2Q1Q0`，则 $Q3^* = S2 + S3 + S4 + S5$ ， $Q2^* = S1 \cdot J\text{-Instr} + S1 \cdot I\text{-Load} + S1 \cdot B\text{-Instr} + S1 \cdot S\text{-Instr}$ ， $Q1^* = S1 \cdot R\text{-Instr} + S1 \cdot I\text{-Arith} + S1 \cdot U\text{-auipc} + S1 \cdot U\text{-lui} + S1 \cdot R\text{-jalr} + S1 \cdot S\text{-Instr} + S1 \cdot B\text{-Instr}$ ， $Q0^* = S1 \cdot R\text{-jalr} + S1 \cdot I\text{-Load} + S1 \cdot B\text{-Instr} + S0 + S5$ ，根据时钟信号 `clk`，复位信号 `rst`，利用 D 触发器实现该时序电路(有限状态机)。利用当前状态 `Q3Q2Q1Q0`，根据表 7-1 生成 `PCWr`、`MARWr`、`IRWr`、`MDRWr`、`ALUoutWr`、`CCWr`、`RegWr`、`MemWr`、`MemEn`、`MemtoReg` 以及三态门控制信号 `PCout` 和 `MARout` 等控制信号设计电路。利用当前状态 `Q3Q2Q1Q0` 和指令类型根据表 7-3 和表 7-4 生成 `MemOp`、`NPCASrc` 和 `NPCBSrc` 控制信号设计电路。

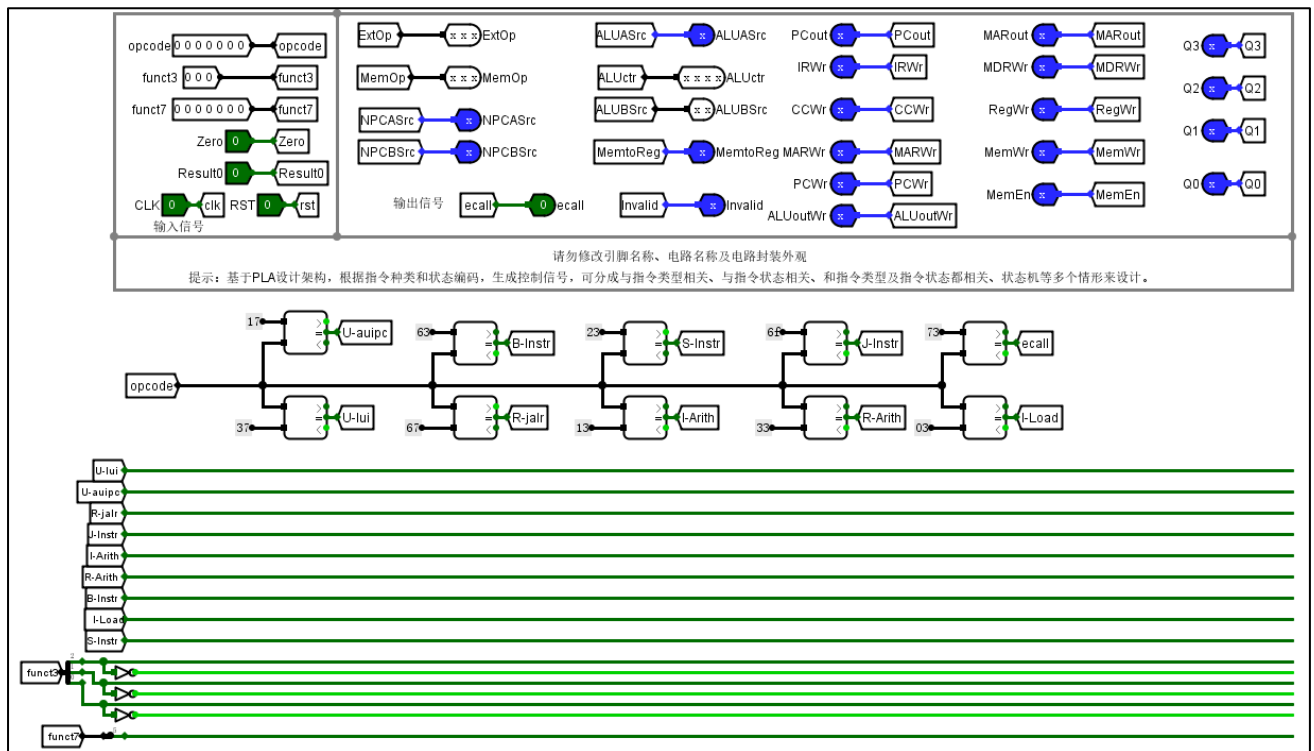


图 7-4 控制器电路引脚布局图

2) 仿真测试并封装子电路。根据 RV32I 中给出的 37 条目标指令的操作码和功能码，逐条输入指令，根据指令执行周期，单击输入时钟周期信号，验证输出控制信号的正确性，保存电路设计文件 lab7.1.circ。

2、多周期 CPU 设计实验

为了保证 CPU 每次重启时，都能从相同的初始状态开始执行程序，需要设计一个复位信号 **Reset**，用来初始化 CPU 中各个部件的控制信号，设置 PC 寄存器的起始地址。

根据图 7-1 所示的多周期 CPU 原理图，在单周期 CPU 设计的基础上增加指令寄存器 IR、存储器地址寄存器 MDR、ALU 结果寄存器 ALUout、存储器数据寄存器 MDR 和状态寄存器 CC 五个寄存器；增加两个三态门。CPU 中主要模块在前期的实验中已经完成，在实验 3 中实现了寄存器堆模块，实验 4 中实现了 ALU 模块，实验 5 中实现了数据存储器、取指令部件，本次实验完成了控制器模块。

可利用前面实验已经完成的寄存器堆、取指令部件、ALU、控制器等电路，连接生成多周期处理器，最终可在该处理器上运行 RV32I 的程序来验证处理器设计的正确性。

为了验证程序执行周期数，在设计中添加一个时钟周期计数器，用来记录程序执行的总周期数。

实验步骤如下。

1) 实现主电路。在 Logisim 中打开实验模版文件 lab7.2.circ，其引脚布局图如 7-5 所示，在 logisim 的 project 菜单下添加 logisim 库文件，把实验 3、实验 4 和实验 5 的设计文件添加到 lab7 的项目中，将库文件

中的设计电路复制到相应的子电路中。然后在工作区中放置子电路，如取指令模块、数据存储器 RAM、数据通路 IDU 和控制器部件，以及输入输出引脚和隧道，如图 7-6 所示的布局引脚。根据模块中定义信号进行连接。设置指令存储器和数据存储器的属性，片选信号都定义为高电平有效。指令存储器的地址位宽设置为 16 位，数据位宽设置为 32 位。

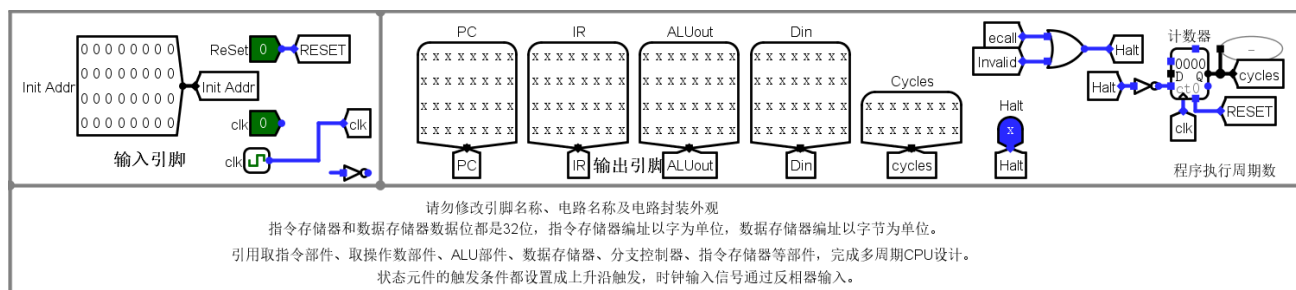


图 7-5 多周期 CPU 布局图

注意：RISC-V 架构定义 0 号寄存器的值始终为 0，需要检查寄存器堆中 0 号寄存器是否已经做了硬件设计处理。

2) 测试指令验证。

为了测试系统整体功能和主要指令的执行情况，实验提供了一个汇编语言测试程序 lab7.2.asm，用来检查加减运算、移位、访存、跳转等指令的功能，如果指令执行结果验证通过，则在寄存器堆的 10 号寄存器中写入“00c0ffee”；否则在 10 号寄存器写入“deaddead”，并在 3 号寄存器中写入当前测试指令的序号。

```
.text
main:
test_1:          #x0 register,addi,xor,lui,bne test
    lui  x0, 0x80000
    addi x0, x0, 0x7ff
    xor  x1, x1, x1
    addi x3, x0, 1      #R[3]=1
    bne  x0, x1, fail
test_2:          #add test
    lui  x1, 0x80000
    lui  x2, 0xffff8
    addx14, x1, x2
    lui  x7, 0x7fff8
    addi x3, x0, 2      #R[3]=2
    bne  x14, x7, fail
test_3:          #and test
    lui  x1, 0xff01
    addi  x1, x1, -16   # 0ff00ff0
    lui  x2, 0xf0f0f
    addi  x2, x2, 240   # f0f0f0f0
    andx15, x1, x2
```

```

lui x7,0xf00
addi x7,x7,0xf0
addi x3,x0,3          #R[3]=3
bnex15,x7, fail
test_4:                #sll test
addi x1,x0,1
addi x2,x0,63
sll x16,x1,x2
lui x7,0x80000
addi x3,x0,4          #R[3]=4
bnex16,x7, fail
test_5:                #sra test
lui x1,0x80000
addi x2,x0,14
sra x17,x1,x2
lui x7,0xfffe0
addi x3,x0,5          #R[3]=5
bnex17,x7, fail
test_6:                #slti test
lui x1,0x80000
slti x18,x1,2047
addi x7,x0,1
addi x3,x0,6          #R[3]=6
bnex18,x7, fail
test_7:                #sltiu test
lui x1,0x80000
sltiu x19,x1,2047
addi x7,x0,0
addi x3,x0,7          #R[3]=7
bne x19,x7, fail
test_8:                #load,store test
auipc x1, 0
addi x1,x1,204        #buffer
addi x2,x0,0xee
sb x2,0(x1)
addi x2,x0,0xff
sb x2,1(x1)
addi x2,x0,0xc0
sb x2,2(x1)
addi x2,x0,0x00
sb x2,3(x1)
lb x20,1(x1)
addi x7,x0,-1
addi x3,x0,8          #R[3]=8

```

```

        bne    x20,x7, fail
    lhu x21,0(x1)
        lui    x7,16    #10
        addi x7,x7,-18    #0xffee
        addi x3,x0,9      #R[3]=9
        bne    x21,x7, fail
    sh    x21,4(x1)
    lh    x22,2(x1)
        addi x7,x0,0xc0
        addi x3,x0,10      #R[3]=10
        bne    x22,x7, fail
    sh    x22,6(x1)
    lw    x23,0(x1)
        lui    x7,0xc10
        addi x7,x7,-18    #c0ffee
        addi x3,x0,11      #R[3]=11
        bne    x23,x7, fail
    lw    x24,4(x1)
        lui    x7,0xc10
        addi x7,x7,-18    #c0ffee
        addi x3,x0,12      #R[3]=12
        bne    x24,x7, fail
test_9:                                #jalr test
    auipc x25, 0
    jalr  x25, x25,24                # test_9_2
test_9_1:
    add x0, x0,x0
    jal x0, fail
test_9_2:
    add x0,x0,x0
    jal x0, fail
auipc x7,0
addi x7,x7,-21                      #test_9_1
addi x3,x0,13                      #R[3]=13
bne x25,x7,fail
pass:
    lui    x10,0xc10
    addi    x10,x10,-18    # R[10]=0X00c0ffee
    ecall                                #opcode=0x73, 程序中止执行
fail:
    lui    x10,0xdeade
    addi    x10,x10,-339    # R[10]=0Xdeaddead
    ecall                                # opcode=0x73, 程序中止执行

```

可以在 RARS 中调试该程序 lab7.2.asm，并导出 16 进制程序代码。为了简化实验步骤，本次实验提供

了生成的机器代码数据文件 lab7.2.hex，按照字节位置分拆成 4 个文件 lab7.2.hex0~lab7.2.hex3，lab7.2.hex0 表示最低字节的机器代码，lab7.2.hex3 表示最高字节的机器代码。在存储器 RAM0~RAM3 中分别加载 lab7.2.hex0~lab7.2.hex3 文件，按照时钟单步执行，观测实验结果，写出寄存器堆和存储器里的数据，验证电路功能和程序执行总周期数，保存电路文件为 lab7.2.circ。

3、测试程序验证。

在多周期 CPU 设计中，程序存储器和数据存储器合二为一，使用统一的地址编码。因此在编写程序时，可以设置数据段，对数据段进行读写操作。

例如计算累计和程序设置如下，入口参数 n 存放在数据段地址 0 单元中，累加和 S 保存在数据段地址 4 单元中。程序运行过程中参数 n、循环变量 i、累加和 S 分别存放在寄存器 a0、a2、a3 中，对应的汇编语言源程序如下：

```
#计算累加和 RV32I 汇编程序
.text
main:
    la t0,n           # 获取标号 n 的地址，la 是伪指令，等价于 auipc t0,标号 n 地址高 20 位和
addi t0,t0,标号 n 地址低 12 位这两条指令
    lw a0,0(t0)       # 从数据存储器的地址单元中读取参数 n 到寄存器 a0;
    beq a0,x0,fail
    ori a2,x0,1       # 循环变量 i，存放在 a2，初值为 1
    xor a3,a3,a3      # 累计和存放在 a3，初值为 s=0
loop:
    add a3, a3, a2     # 将 a3=a3+i
    beq a2, a0, finish # 若 i=n，则跳出循环
    addi a2, a2, 1     # i++
    jal x0, loop       # 无条件跳转到 loop 执行
finish:
    sw a3, 4(t0)       # 将累加结果保存到数据存储器地址 0x2004 单元
pass:
    lui    a0,0xc10
    addi   a0,a0,-18   # 00c0ffee
    ecall
fail:
    lui    a0,0xdeade
    addi   a0,a0,-339  # deaddead
    ecall

.data           # 在 RARS 中可设置数据段从 0x2000 开始，则数据段与代码段之间间隔
(8192-68)/4-1 个字长,字长 32 位
n: .word 100     #参数 n
result: .word 0  #计算结果
```

在 RARS 中调试时,为了模拟多周期 CPU 的系统状态中,需配置 RARS 的内存分配模式,在菜单 Setting 中的 Memory Configuration 选项设置为“Compact, Text at Address 0”,如图 7-6 所示,这样代码段的起始地址就从 0 开始,数据段的起始地址从 0x00002000 开始。

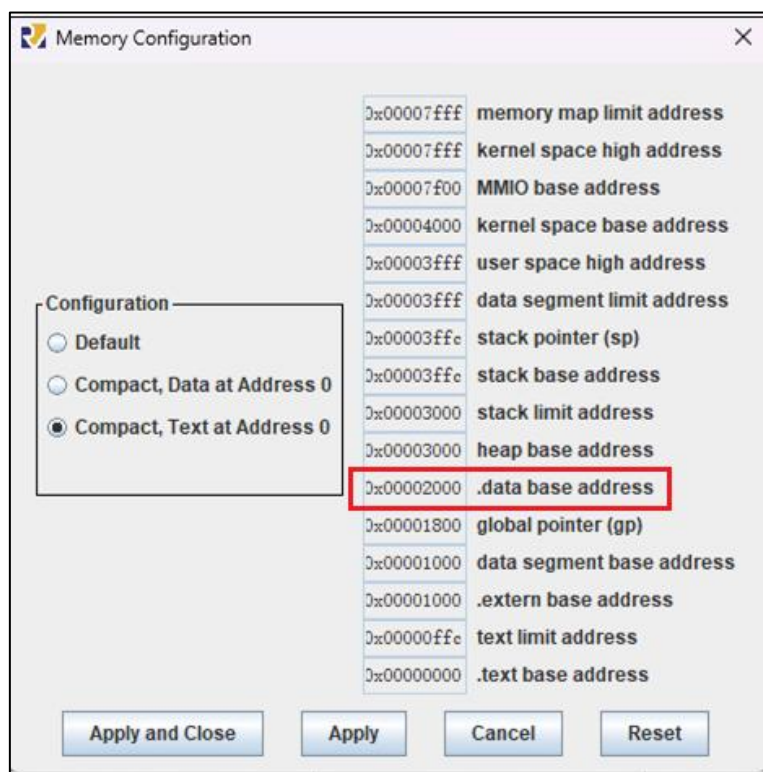


图 7-6 RARS 中 Memory Configuration 选项设置

可在 RARS 数据段 (Data Segment) 的起始地址 0x00002000 中,修改参数 n,重新执行程序,验证计算结果。

验证通过后,导出机器代码,在 RARS 的 File 菜单中点击“Dump Memeory To File”按钮,选择导出格式为 16 进制文本格式 (Hexadecimal Text),分别将代码段代码段.text (0x00000000-0x00000040)和数据段.data (0x00002000-0x00002ffc)导出,如图 7-7 所示。点击 Dump to File 按钮,分别设置导出文件名为 sum-whole-t.hex 和 sum-whole-d.hex,将这两个文件分别按照字节拆分成 4 个文件 sum-whole-t0.hex~sum-whole-t3.hex 和 sum-whole-d0.hex~sum-whole-d3.hex。sum-whole-t.hex 文件长度是 68 个字节,数据段的内容从存储地址 8192 (0x2000) 开始,因此在代码段和数据段之间相隔 $8191 (0x2000-1) - 68 (0x11*4) = 8120$ 个字节,因此把代码段 4 字节的文件拆分成单字节文件后,需要在单字节文件的后面插入 $8120/4 = 2030$ 个字节空白数据,然后再插入相应字节对应的数据段单字节文件中的有效数据,并在首行插入“V2.0 raw”,保存代码段文件。

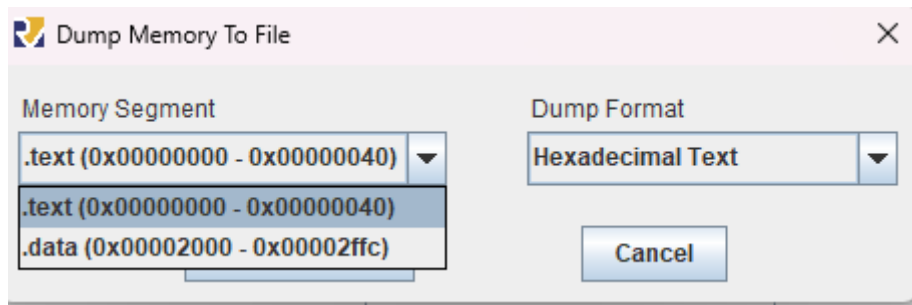


图 7-7 RARS 中导出代码段和数据段的 16 进制格式的文件

打开 sum-whole-t0.hex，其内容如下所示：

```
v2.0 raw
97
93
03
63
13
b3
b3
63
13
6f
23
37
13
73
37
13
73
00
2030*0
64
```

在多周期 CPU 的数据存储器 RAM3~RAM0 中分别加载机器代码镜像文件 sum-whole-t3.hex~sum-whole-t0.hex，启动时钟信号，执行程序。在数据段的 0x2004 地址处，观察输出结果，验证 CPU 设计的正确性。

同理可验证冒泡排序程序 bubble-sort.asm，汇编语言程序参考代码如下：

#冒泡排序算法 RV32I 汇编程序

.text

la sp, a

lw a0,0(sp) #a0,保存排序数量 n,待排序的数字个数 n 存在数据段初始地址

addi a1,x0,1 #a1, 保存常量 1

add a2,a0,x0 #a2, 保存 i, 初始值为 i=N

L1:

```

    add a3,a1,x0 #a3, 保存 j, 初始值为 j=1
L2:
    slli a4,a3,2    # a4 保存 a[j]地址
    add a4,a4,sp
    lw  a6,0(a4)     #读取第 j 个元素
    lw  a7,4(a4)     #读取第 j+1 个元素
    bgeu a7,a6,L4    #a[j]>=a[j+1] 跳转
    sw  a7,0(a4)     #交换存储
    sw  a6,4(a4)     #交换存储
L4:
    add a3,a3,a1     #j=j+1
    bltu a3,a2,L2    # if j<i then 循环 读取两个元素比较
L3:
    sub a2,a2,a1     #i--
    bne a2,a1,L1     #if i>1 then 循环 else 则结束
finish:
    jal x0, finish

```

.data

a:.word 0xa 0x8 0x41 0x2 0x12 0x36 0x6 0x9 0x5 0x5b 0x7

在 RARS 中编译、执行，导出代码段和数据段，并按字节进行拆分，然后合并代码段和数据段，生成机器代码镜像文件，加载到多周期 CPU 中进行验证测试，导出排序结果。

4、官方测试验证。

打开实验 7 提供的官方测试集文件 `testcase.zip`，加载每条指令的测试文件，验证实验结果，保存电路文件。

四、思考题

1. 修改累加和和冒泡排序中参数，验证指令执行的总周期。
2. 如果不使用投机计算，多周期 CPU 设计原理图，该如何修改？每种类型的指令执行时钟周期数增加了多少？
3. 如果在多周期 CPU 中增加中断指令，执行该指令时，暂停指令运行，查看程序运行状态，并可继续执行下一条指令，则设计电路中需要做哪些修改。
4. 如果在取指令阶段必须更新 PC+4 到 PC 寄存器中，则如何修改多周期 CPU 设计才能保证跳转和分支指令的可靠执行？
5. 如果需要进行 5 级流水线 RV32I CPU，则在多周期 CPU 基础上如何修改？